

NPTEL

Variants of Gradient Descent and Momentum Based Optimization Techniques

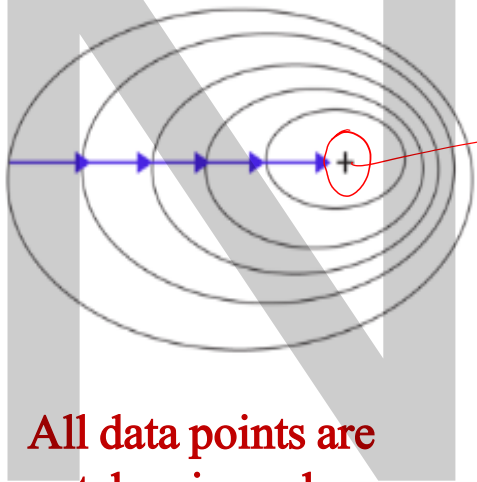
Content

- Batch, Mini Batch and Stochastic Gradient Descent
- Stochastic Gradient Descent with Momentum
- Nesterov Accelerated Gradient (NAG)
- Adaptive Gradient Algorithm
- RMSProp Optimizer
- Adam Optimizer
- AdaDelta Optimizer

Variants of Gradient Descent

$$\theta_{new} = \theta_{old} - \eta \nabla_{\theta} L(\theta)$$

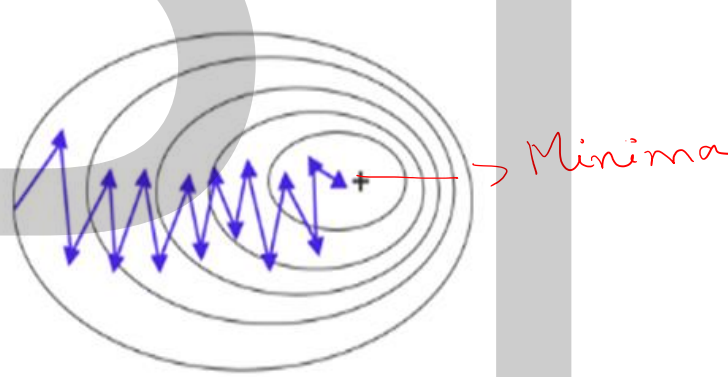
Batch Gradient Descent



All data points are taken in each iteration

Stable, but slow convergence for large dataset.

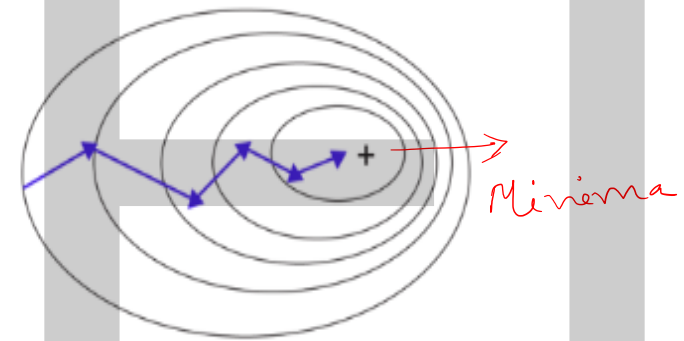
Stochastic Gradient Descent



Single data point is taken in each iteration

Fast convergence, but noisy.

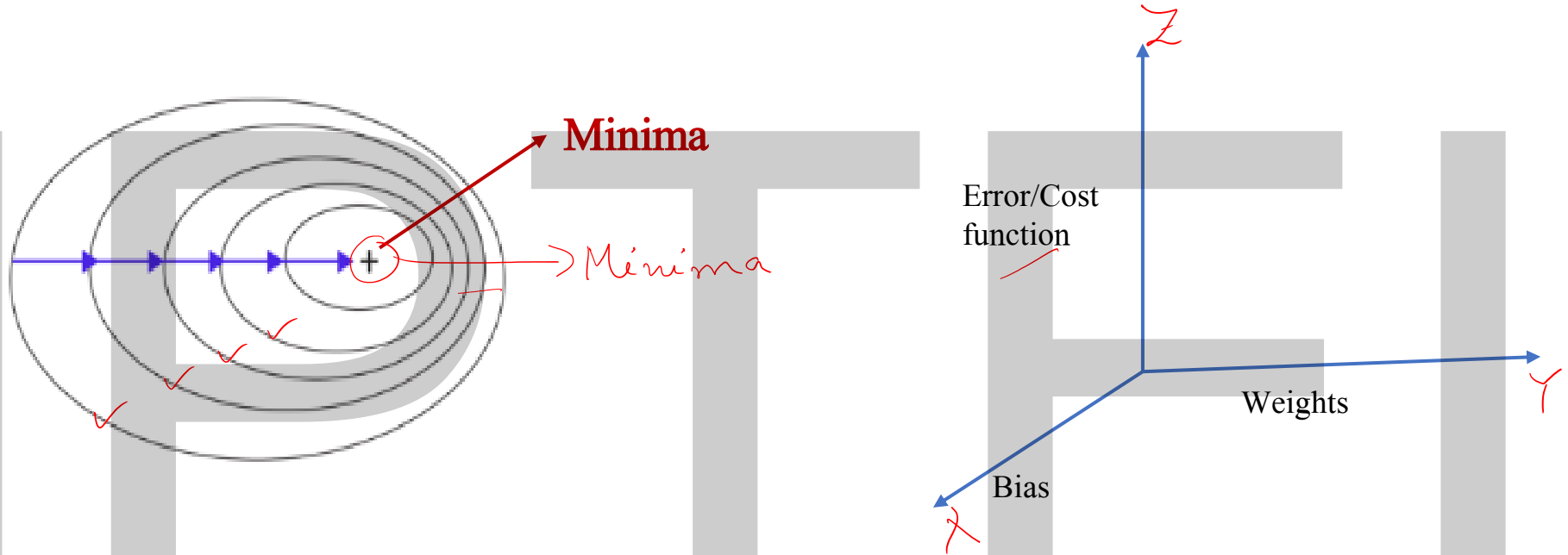
Minibatch Gradient Descent



Small subset of data points are taken in each iteration

In between batch Gradient Descent and Stochastic Gradient Descent

Why are we using contours to explain gradient descent?



- ✓ 3D surfaces can be visualized using 2D plots where the contours represent points having same z value.
- ✓ Closely spaced contours represent steep slope in that direction

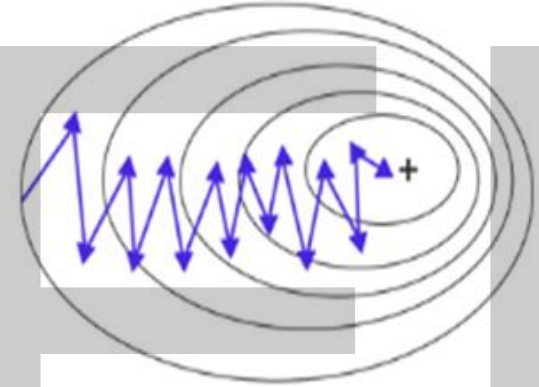
Stochastic Gradient Descent (SGD)

‘Stochastic’ refers to a system or process that is linked with a random probability. In SGD, this randomness is introduced in the way the gradient is calculated.

SGD works by randomly sampling one data point at each iteration, and then using the gradient of those data points to update the model parameters

Why is SGD Noisy?

- ✓ In SGD, gradient is calculated only for a single point, hence it is not the true gradient.
- ✓ It is a point estimation.
- ✓ **This makes it noisy**



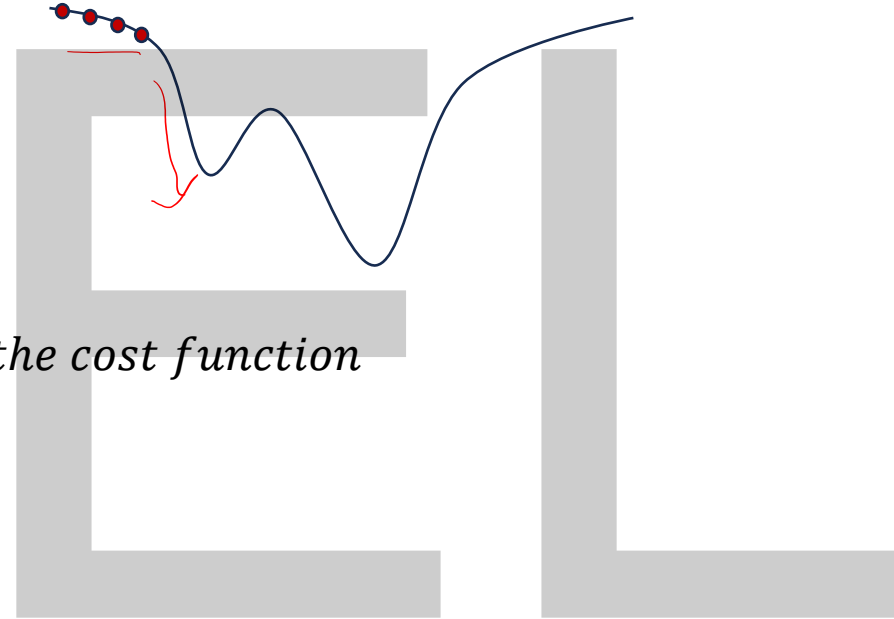
Observations about Gradient Descent

Long time taken by Gradient Descent to navigate regions of lesser slope.

- ✓ Gradients are less in the regions where slope is gentle.
- ✓ This results in smaller weight updation.

$$w_{t+1} = w_t - \eta g_t$$

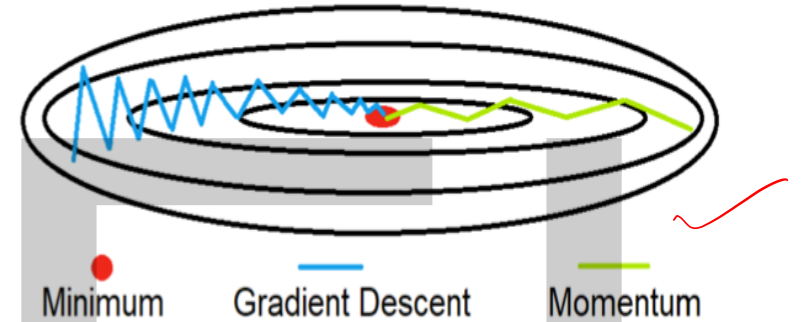
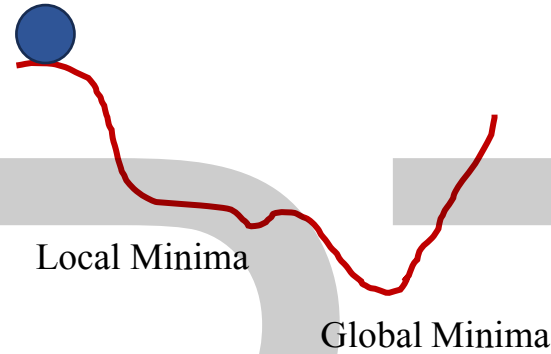
g_t = Gradient of the cost function



Momentum based Optimizers

- ✓ Similar to a ball rolling from the top of a hill.
- ✓ Builds inertia in the search direction to overcome local minima and oscillation of noisy gradients.

A ball rolling from the top of a hill gathers enough momentum to overcome a plateau region and make it to a global minima instead of getting stuck at a local minima.



Momentum adds history to the parameter updates thus accelerating the optimization process.

Update rule for SGD with Momentum

$$\underline{update}_t = \gamma(\underline{update}_{t-1}) + \underline{\eta g_t}$$

$$\underline{w_{t+1}} = \underline{w_t} - \underline{update_t}$$

γ : Hyperparameter that decides how much momentum to be added.(ranges between 0 and 1)

SGD without Momentum

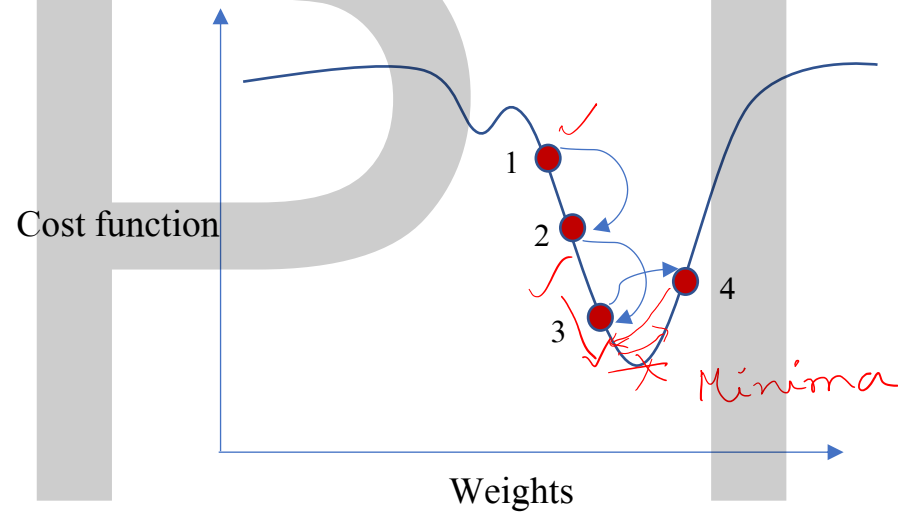
$$w_{t+1} = w_t - \eta g_t$$

Similarly biases also will get updated

Image Ref:

<https://optimization.cbe.cornell.edu/index.php?title=Momentum>

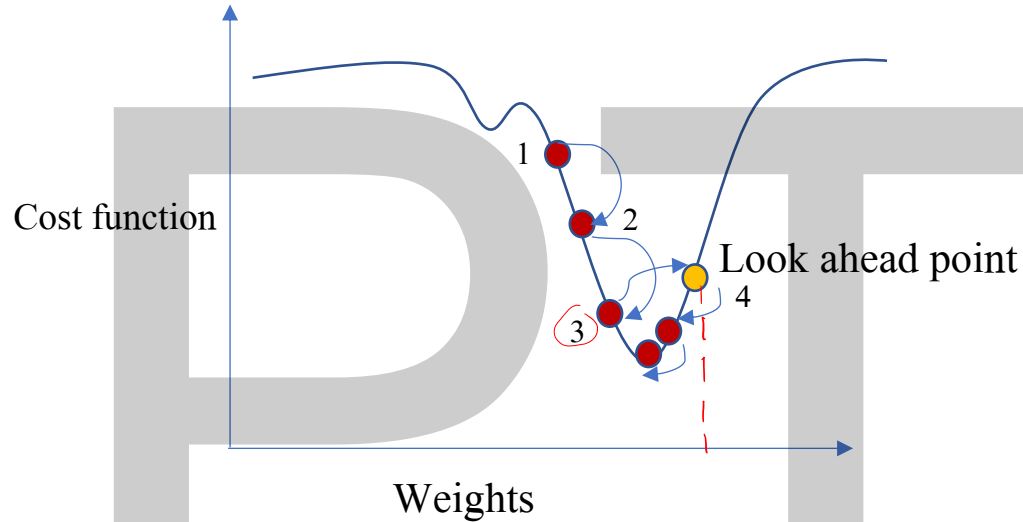
Observation about Gradient Descent with Momentum



May lead to oscillations before convergence

Nesterov Accelerated Gradient (NAG)

Core idea:
Look ahead before
you jump.



Update rule in Gradient Descent with momentum:

$$\begin{aligned} \text{update}_t &= \gamma(\text{update}_{t-1}) + \eta g_t \\ w_{t+1} &= w_t - \text{update}_t \end{aligned}$$

In NAG, we calculate the gradient at the look ahead point

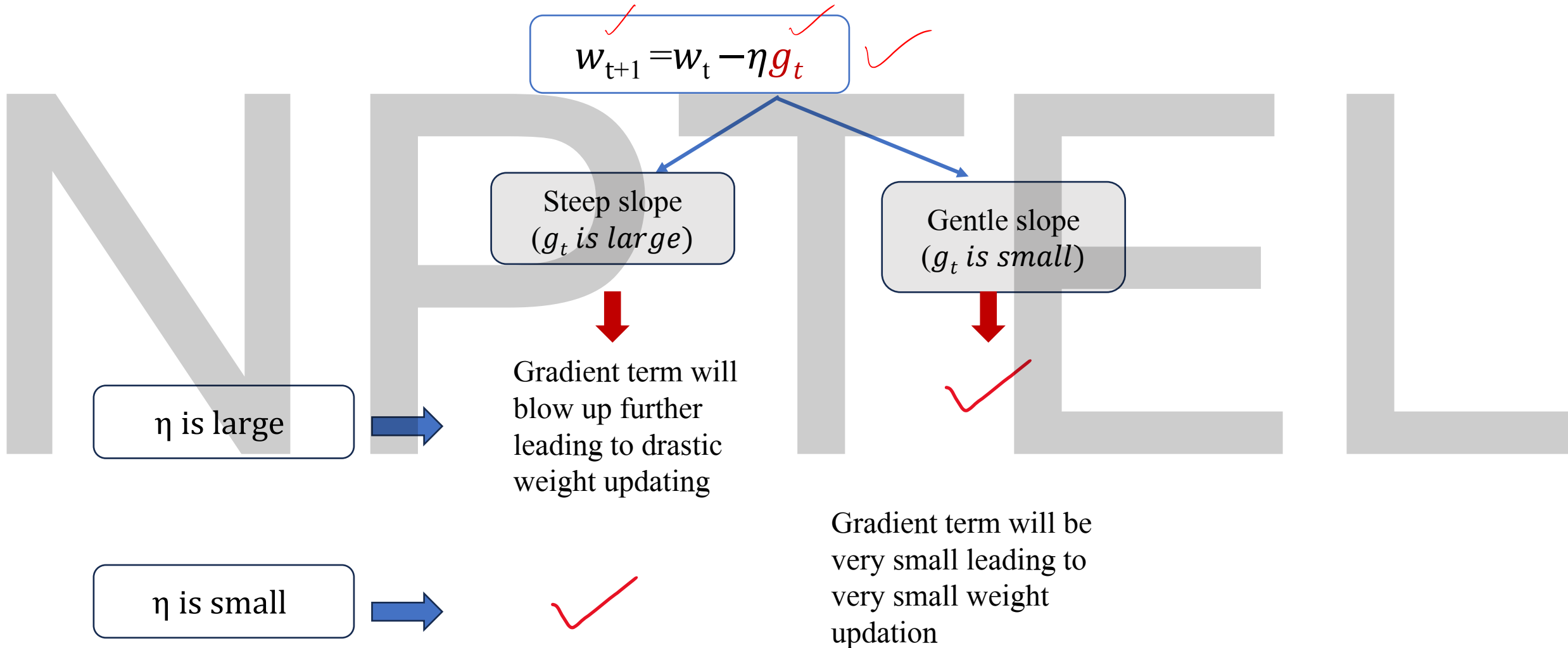
$$w_{\text{look ahead}} = w_t - \gamma(\text{update}_{t-1}) \text{ [partially updated weight]}$$

Update rules of NAG:

$$\begin{aligned} \text{update}_t &= \gamma(\text{update}_{t-1}) + \eta(g_t)_{w(\text{look ahead})} \\ w_{t+1} &= w_t - \text{update}_t \end{aligned}$$

Similarly biases also will get updated

Challenges with a Single Learning Rate



- ✓ It will be good if we can have learning rates which can adjust to gradients.
- ✓ Gradients change after every iteration

Time-based decay

$$\eta_t = \frac{\eta_0}{(1 + kt)}$$

η_0 is the initial learning rate

t = current step number

η_t = Learning rate at time step t

Both η_0 and k are hyperparameters (k is a fixed decay rate, typically 0.1 or 0.01)

As the number of steps grow learning rate decreases.

Exponential decay

$$\eta_t = \eta_0 * e^{-kt}$$

Gradient Descent with Adaptive Learning Rate

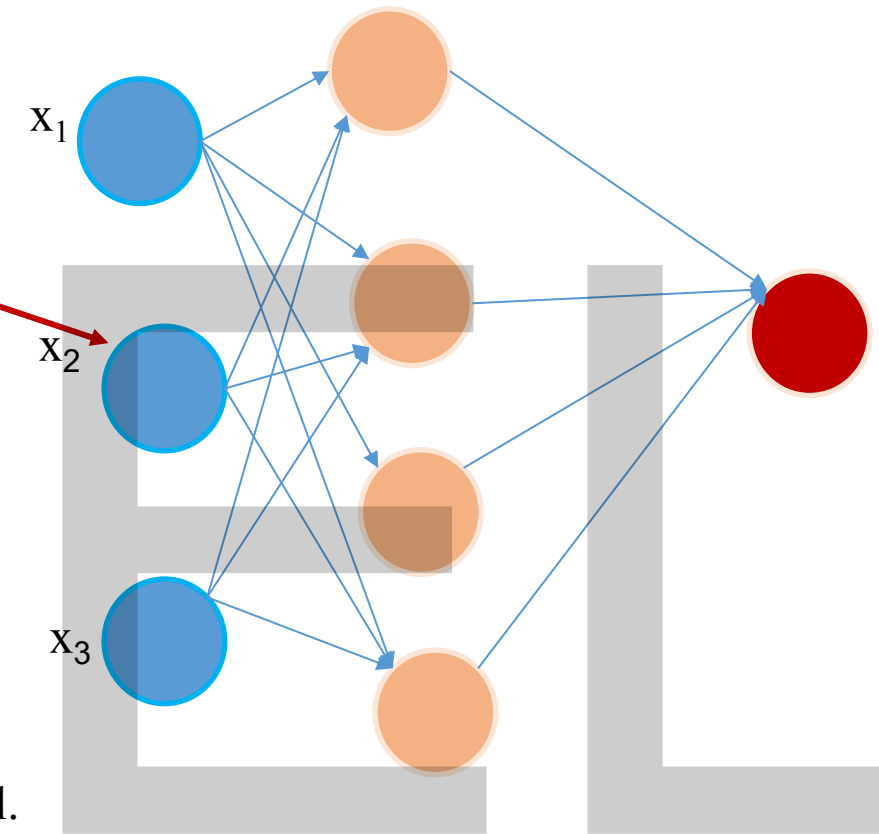
Sparse Dataset

Users	Movies watched				Ratings	Age
	Avengers	Inception	Star wars	Lord of the Rings		
A	0	0	1	0	5	55
B	1	0	0	0	4	43
C	0	0	0	0	-	25
D	1	1	0	0	4	40

$$X=[x_1, x_2, x_3]$$

Gradient of the cost function w.r.t weights associated with x_2 will be very small.
The parameter update will be minimum at each time step.

If we have a dense dataset, then parameter update will be large for each timestep



Adaptive Gradient (AdaGrad)

Core Idea:

Tunes the learning rate based on the previous updates.

- If previous updates are high, learning rate will decrease
- If previous updates are less, learning rate doesn't decrease to a great extent

Update rule for AdaGrad:

$$v_t = v_{t-1} + g_t^2$$

Running sum of the history of the updates

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} * g_t$$

Learning rate is decreasing

Learning rate decays slowly for sparse dataset

Learning rate decays aggressively for dense features with time step.

But, due to this the optimizer may get stuck near to the point of convergence

RMSProp Optimizer

Update rule for RMSProp:

$$v_t = \beta v_{t-1} + (1-\beta)g_t^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} * g_t$$

$$\beta = 0.95$$

$$\begin{aligned} v_t &= 0.95v_{t-1} + (1-0.95)g_t^2 \\ &= 0.95v_{t-1} + (0.05)g_t^2 \end{aligned}$$

Adagrad:

$$\begin{aligned} v_t &= v_{t-1} + g_t^2 \\ w_{t+1} &= w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} * g_t \end{aligned}$$

- ✓ RMSProp keeps track of an exponentially decaying average of the squared gradients.
- ✓ Thus, RMSProp overcomes the problem of aggressive learning rate decrease in Adagrad

$$v_0 = \beta v_{0-1} + (1-\beta)g_0^2 \quad \beta = 0.95$$

$$v_0 = (1-0.95)g_0^2 = \underline{0.05}g_0^2 \quad \checkmark$$

$$\begin{aligned} v_1 &= \beta v_0 + (1-\beta)g_1^2 \\ &= 0.95 \times 0.05g_0^2 + \underline{0.05}g_1^2 \\ &= \underline{0.0475}g_0^2 + 0.05g_1^2 \end{aligned}$$

$$\begin{aligned} v_2 &= \beta v_1 + (1-\beta)g_2^2 \\ &= 0.95[0.0475g_0^2 + 0.05g_1^2] + 0.05g_2^2 \\ &= \underline{0.045125}g_0^2 + \underline{0.0475}g_1^2 + 0.05g_2^2 \end{aligned}$$

Adam Optimizer

Adam (Adaptive Moment Estimation) optimizer combines the advantages of Momentum and RMSprop techniques to adjust learning rates during training.

Update Rule for Adam Optimizer:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\sqrt{\widehat{\mathbf{v}}_t} + \epsilon} * \widehat{\mathbf{m}}_t$$

Where

$$\widehat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$$

$$\widehat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}$$

Gradient descent

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta * \mathbf{g}_t$$

$\downarrow$
 $\widehat{\mathbf{m}}_t$

This is same as momentum based gradient descent

Similar to RMSProp

Adam is the default choice for most real world problems

AdaDelta Optimizer

- ✓ This optimizer avoids having an initial learning rate
- ✓ No hyper parameter tuning is required

Update rule for AdaDelta:

$$\begin{aligned} \mathbf{w}_{t+1} &= \mathbf{w}_t + \Delta \mathbf{w}_t \\ \Delta \mathbf{w}_t &= - \frac{\sqrt{u_{t-1} + \epsilon}}{\sqrt{v_t + \epsilon}} \cdot \mathbf{g}_t \\ v_t &= \beta v_{t-1} + (1 - \beta) \mathbf{g}_t^2 \\ u_t &= \beta u_{t-1} + (\Delta \mathbf{w}_t)^2 \\ \mathbf{w}_{t+1} &= \mathbf{w}_t - \frac{\sqrt{u_{t-1} + \epsilon}}{\sqrt{v_t + \epsilon}} \mathbf{g}_t \end{aligned}$$

Let us look at time step $t=2$

$$w_3 = w_2 + \Delta w_2$$

$$\Delta w_2 = - \frac{\sqrt{u_1 + \epsilon}}{\sqrt{v_2 + \epsilon}} g_2$$

$$v_2 = \beta v_1 + (1 - \beta) g_2^2$$

$$u_1 = \beta u_0 + (\Delta w_1)^2$$

$$v_2 > u_1$$

- ✓ *Effective learning rate is small*
- ✓ *Δw_2 is small, when gradient is steep.*

Near to regions of gentle slope, $u > v$

- ✓ *Effective learning rate is more near regions of gentle slope*

AdaDelta dynamically tunes the effective learning rate without having an initial learning rate.

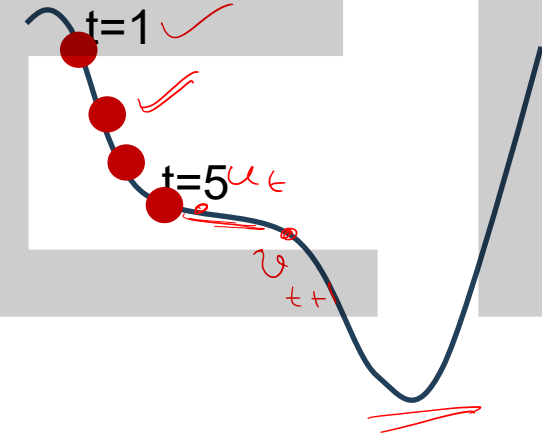
$$w_{t+1} = w_t + \Delta w_t$$

$$\Delta w_t = - \frac{\sqrt{u_{t-1} + \epsilon}}{\sqrt{v_t + \epsilon}} g_t$$

$$v_t = \beta v_{t-1} + (1 - \beta) g_t^2$$

$$u_t = \beta u_{t-1} + (\Delta w_t)^2$$

$$w_{t+1} = w_t - \frac{\sqrt{u_{t-1} + \epsilon}}{\sqrt{v_t + \epsilon}} g_t$$



NPTREL

Thank You